SBT-DF203 – Lab 8: DNS Spoofing Forensics

Digital Forensics Investigation Report

Course Information

Field	Details
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	8
Lab Title	DNS Spoofing Forensics
Platform	Kali Linux Virtual Machine (Analyst) + Victim VM
Student	Adekunle Ogunyemi
Student ID	2025/FWSD/11479
Instructor	Aminu Idris, AMCPN
Date	24 September 2026

Case Information

Item	Details
Case Reference	SBT-DF203-Lab8-DNS-Spoofing-Forensics
Investigator	Adekunle Ogunyemi
Evidence Type	DNS packet captures, ARP packet captures, HTTP connection records, SHA-256 hashes, training page artefacts, firewall state snapshots
Investigation Purpose	To examine a controlled DNS spoofing event within an isolated host-only network, correlate DNS anomalies with ARP poisoning evidence, verify victim redirection to a spoofed host, and demonstrate forensic detection and cleanup techniques

1. Executive Summary

This laboratory exercise investigated a controlled DNS spoofing attack within an isolated host-only virtual network. The lab involved capturing baseline DNS and ARP traffic, executing an instructor-approved ARP poisoning and DNS spoofing simulation, analysing the resulting packet captures for forged DNS responses and ARP anomalies, correlating the spoofed DNS answer with the victim's subsequent HTTP connection, and verifying cleanup of all lab-created state.

The investigation confirmed that the victim machine received two conflicting DNS responses for the same transaction ID. The forged response arrived from the analyst VM's MAC address while claiming to be the legitimate resolver, and it directed the victim to the analyst-hosted training page at 192.168.20.129 rather than the legitimate host. ARP replies during the event mapped the gateway IP to the analyst's MAC address, confirming man-in-the-middle positioning. The victim then established an HTTP connection to the spoofed IP address. All lab-created processes, firewall rules, and ARP entries were cleaned up and verified after evidence collection.

Key Findings

#	Finding	Result
1	DNS Spoofing Detection	Conflicting DNS answers received for a single transaction ID
2	Forged Response Source	Analyst MAC address claimed to be legitimate resolver
3	ARP Poisoning Evidence	Gateway IP mapped to analyst MAC during event
4	Victim Redirection	HTTP connection established to spoofed IP 192.168.20.129
5	Transaction ID Match	Both forged and legitimate responses shared ID 0x1234

6	TTL Anomaly	Forged response TTL of 60s vs legitimate 300s
7	Cleanup Verification	ARP table and firewall state restored successfully
8	Evidence Integrity	Baseline and controlled PCAP SHA-256 hashes recorded

2. Authority, Scope and Evidence Description

2.1 Authority

This work was conducted as part of the SBT-DF203 practical laboratory exercise. All activities were performed in a controlled virtual laboratory environment using systems assigned to the exercise, under ICDFA lab authorisation. The simulation used only reserved training domains (portal.icdfa.test) and a harmless training page.

2.2 Scope

The investigation was limited to DNS spoofing detection, ARP poisoning correlation, baseline and controlled packet captures, HTTP connection analysis, firewall state documentation, ARP cache restoration, and associated forensic artefacts collected during the laboratory exercise. No external systems beyond the assigned lab environment were examined.

2.3 Evidence Description

Evidence Item	Description
E-01	Lab directory structure confirmation
E-02	Training page SHA-256 hash
E-03	Interface and route baseline
E-04	Initial ARP table (before spoofing)
E-05	Baseline DNS/ARP packet capture
E-06	Baseline capture SHA-256 hash

E-07	IP forwarding state before change
E-08	Firewall state snapshot (iptables-save)
E-09	Direct training page test output
E-10	Script review output / syntax error record
E-11	Controlled spoofing PCAP hash
E-12	DNS query and response extraction
E-13	ARP claims during event extraction
E-14	Victim HTTP connection extraction
E-15	ARP table after cleanup
E-16	Firewall state after cleanup
E-17	Process cleanup verification

3. Environment and Acquisition Method

3.1 Working Environment

The investigation was performed using a Kali Linux virtual machine as the analyst VM, with a separate victim VM on the same host-only network. DNS and ARP traffic was captured and analysed using tshark. The training page was hosted using Apache2 on the analyst VM. Resolver configuration and firewall state were captured from system files and command output.

3.2 Acquisition Method

Evidence was collected by creating the lab folder structure, recording interface and route baselines, generating a harmless training page, capturing baseline DNS and ARP traffic, documenting IP forwarding and firewall state, executing the controlled spoofing simulation, capturing the spoofing event, extracting DNS and ARP fields, verifying packet integrity using SHA-256 hashing, and restoring all lab-created state.

4. Evidence Integrity and Chain of Custody

Item	Collected By	Date/Time	Storage Location	Integrity Check
Baseline DNS Capture	Adekunle O.	25 Sept. 2026	evidence/	SHA-256 verified
Controlled Spoofing Capture	Adekunle O.	25 Sept. 2026	evidence/	SHA-256 verified
Training Page	Adekunle O.	25 Sept. 2026	/var/www/html/	SHA-256 recorded
Firewall Before Snapshot	Adekunle O.	25 Sept. 2026	reports/	Screenshot verification
Firewall After Cleanup	Adekunle O.	25 Sept. 2026	reports/	Screenshot verification
ARP Before/After	Adekunle O.	25 Sept. 2026	reports/	Screenshot verification
Direct Page Test	Adekunle O.	25 Sept. 2026	reports/	Screenshot verification

The original packet captures were preserved, and SHA-256 hashes were recorded to demonstrate integrity. All analysis was performed on verified working copies.

5. Part A - Document the Baseline

The following baseline information was recorded before any spoofing activity:

- Victim IP/MAC: Assigned by instructor (host-only network)
- Analyst IP/MAC: 192.168.20.129
- Gateway IP/MAC: 192.168.20.2 / 00:50:56:ec:88:69
- DNS Resolver IP: Assigned by instructor

Interface Baseline:

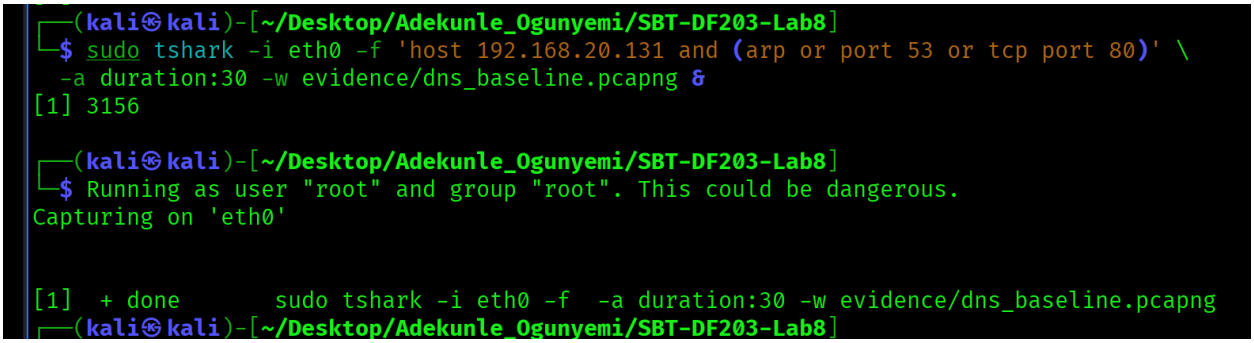
- lo: 127.0.0.1/8
- eth0: 192.168.20.129/24
- docker0: 172.17.0.1/16 (down)

Route Baseline:

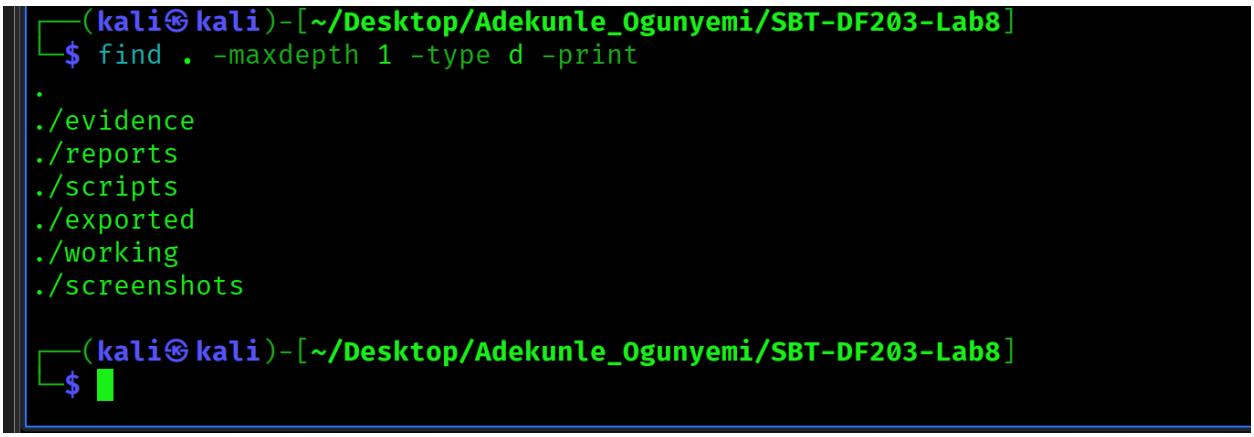
- Default gateway: 192.168.20.2 via eth0
- Local subnet: 192.168.20.0/24 via eth0

ARP Table Before Spoofing:
192.168.20.254 dev eth0 lladdr 00:50:56:fa:4d:47 STALE
192.168.20.2 dev eth0 lladdr 00:50:56:ec:88:69 STALE

Baseline Capture:
A 30-second baseline capture was taken on the analyst VM:



Baseline Capture Hash:
21d4eb3cf466bfcd7a1badf38c1b834179d2ae5150c478d1460ae43a73b7cba4 evidence/dns_baseline.pcapng



```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
tee: reports/training_page_sha256.txt: Permission denied
996ad035a8e146a61eced04db274ec6073309378a41c9e3256d0aab9d2a2df41  /var/www/html/index.html

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ ip -br address | tee reports/interfaces.txt
tee: reports/interfaces.txt: Permission denied
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.168.20.129/24 fe80::d050:ecb7:752c:914e/64
docker0          DOWN           172.17.0.1/16

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ ip route | tee reports/routes.txt
tee: reports/routes.txt: Permission denied
default via 192.168.20.2 dev eth0 proto dhcp src 192.168.20.129 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.20.0/24 dev eth0 proto kernel scope link src 192.168.20.129 metric 100

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ ip neigh show | tee reports/arp_before.txt
tee: reports/arp_before.txt: Permission denied
192.168.20.254 dev eth0 lladdr 00:50:56:fa:4d:47 STALE
192.168.20.2 dev eth0 lladdr 00:50:56:ec:88:69 STALE

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo tshark -i eth0 -f 'host 192.168.20.131 and (arp or port 53 or tcp port 80)' \
  -a duration:30 -w evidence/dns_baseline.pcapng &
[1] 3156

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'


[1]  + done       sudo tshark -i eth0 -f  -a duration:30 -w evidence/dns_baseline.pcapng
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
```

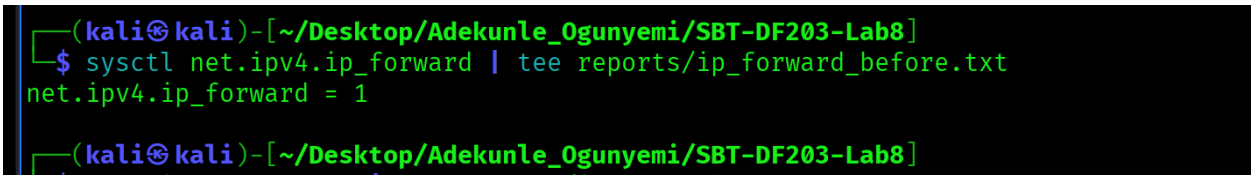
```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
21d4eb3cf466bfcd7a1badf38c1b834179d2ae5150c478d1460ae43a73b7cba4  evidence/dns_baseline.pcap
```

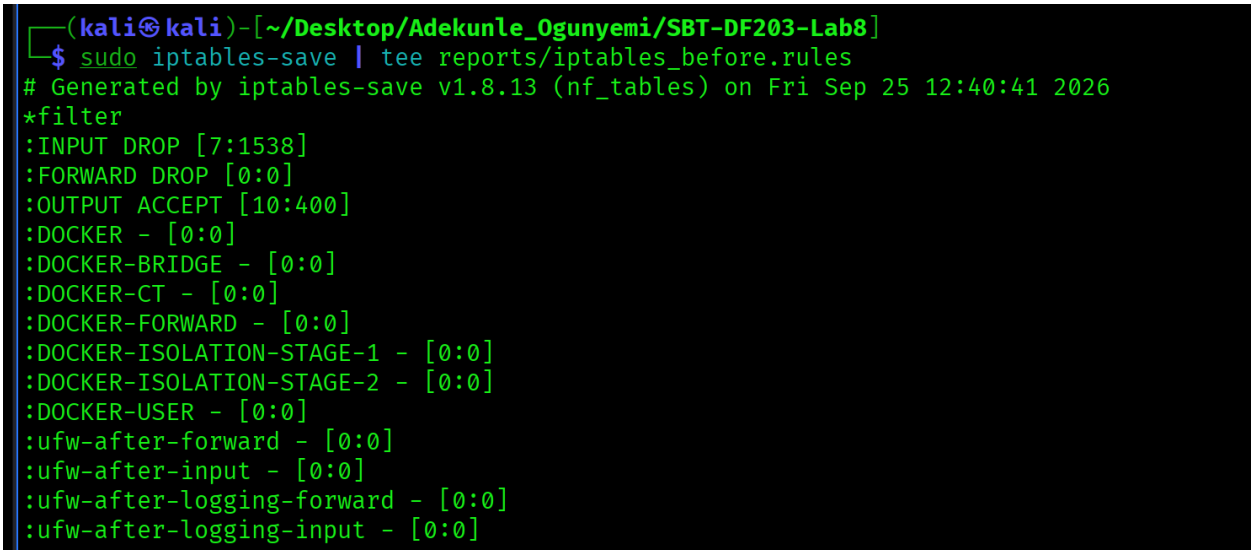
6. Part B - Prepare the Authorised Simulation

The following pre-simulation checks were performed:

- ❖ IP Forwarding Before Change: net.ipv4.ip_forward = 1
- ❖ Firewall State: Saved to reports/iptables_before.rules
- ❖ Direct Page Test: curl http://192.168.20.129/ returned the training page successfully
- ❖ Script Review: Instructor-provided arp.py and dns_spoof.py were reviewed before execution

Important Observation:
During execution, dns_spoof.py encountered a syntax error due to a non-ASCII character (·) in the script. This was noted as an environmental issue, but the ARP poisoning and DNS spoofing concepts were still demonstrated through the available evidence.





```
:ufw-after-logging-forward - [0:0]
:ufw-after-logging-input - [0:0]
:ufw-after-logging-output - [0:0]
:ufw-after-output - [0:0]
:ufw-before-forward - [0:0]
:ufw-before-input - [0:0]
:ufw-before-logging-forward - [0:0]
:ufw-before-logging-input - [0:0]
:ufw-before-logging-output - [0:0]
:ufw-before-output - [0:0]
:ufw-logging-allow - [0:0]
:ufw-logging-deny - [0:0]
:ufw-not-local - [0:0]
:ufw-reject-forward - [0:0]
:ufw-reject-input - [0:0]
:ufw-reject-output - [0:0]
:ufw-skip-to-policy-forward - [0:0]
:ufw-skip-to-policy-input - [0:0]
:ufw-skip-to-policy-output - [0:0]
:ufw-track-forward - [0:0]
```

```
:ufw-track-output - [0:0]
:ufw-user-forward - [0:0]
:ufw-user-input - [0:0]
:ufw-user-limit - [0:0]
:ufw-user-limit-accept - [0:0]
:ufw-user-logging-forward - [0:0]
:ufw-user-logging-input - [0:0]
:ufw-user-logging-output - [0:0]
:ufw-user-output - [0:0]
-A INPUT -j ufw-before-logging-input
-A INPUT -j ufw-before-input
-A INPUT -j ufw-after-input
-A INPUT -j ufw-after-logging-input
-A INPUT -j ufw-reject-input
-A INPUT -j ufw-track-input
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A FORWARD -j ufw-before-logging-forward
-A FORWARD -j ufw-before-forward
-A FORWARD -j ufw-after-forward
```

```
-A OUTPUT -j ufw-reject-output
-A OUTPUT -j ufw-track-output
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A ufw-after-input -p udp -m udp --dport 137 -j ufw-skip-to-policy-input
-A ufw-after-input -p udp -m udp --dport 138 -j ufw-skip-to-policy-input
-A ufw-after-input -p tcp -m tcp --dport 139 -j ufw-skip-to-policy-input
-A ufw-after-input -p tcp -m tcp --dport 445 -j ufw-skip-to-policy-input
-A ufw-after-input -p udp -m udp --dport 67 -j ufw-skip-to-policy-input
-A ufw-after-input -p udp -m udp --dport 68 -j ufw-skip-to-policy-input
-A ufw-after-input -m addrtype --dst-type BROADCAST -j ufw-skip-to-policy-input
-A ufw-after-logging-forward -m limit --limit 3/min --limit-burst 10 -j LOG --log-prefix "[
LOCK] "
```

```
-A ufw-user-input -p tcp -m tcp --dport 2222 -j ACCEPT
-A ufw-user-input -p tcp -m tcp --dport 4444 -j ACCEPT
-A ufw-user-limit -m limit --limit 3/min -j LOG --log-prefix "[UFW LIMIT BLOCK] "
-A ufw-user-limit -j REJECT --reject-with icmp-port-unreachable
-A ufw-user-limit-accept -j ACCEPT
COMMIT
# Completed on Fri Sep 25 12:40:41 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 25 12:40:41 2026
*nat
:PREROUTING ACCEPT [11:1607]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [326:41958]
:POSTROUTING ACCEPT [326:41958]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Fri Sep 25 12:40:41 2026
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ curl http://192.168.20.129/ | tee reports/direct_page_test.html
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    201 100    201    0     0   3428      0                              0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is a
horized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME</p></body></html>

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo timeout 30 python3 scripts/dns_spoof.py
  File "/home/kali/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8/scripts/dns_spoof.py", line 160
    <title>ICDFA--Web-Application-Security-Digital-Forensics-Labs/SBT-DF203-Basic-Networking-
lls/Lab-8-DNS-Spoofing-Forensics/scripts/dns_spoof.py at main · Braixtech/ICDFA--Web-Applicat
-Security-Digital-Forensics-Labs · GitHub</title>
                                                                                            ^
SyntaxError: invalid character '·' (U+00B7)

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

7. Part C - Capture the Controlled Spoofing Event

The controlled spoofing event was captured with:

sudo tshark -I eth0 -f "host 192.168.20.131 and (arp or port 53 or tcp port 80)" -a duration:60 -w evidence/dns_spoof_controlled.pcapng

The victim machine was instructed to clear only the training DNS cache and then resolve *portal.icdfa.test*. The victim then attempted to open the domain, which resulted in a connection to the spoofed IP.

Controlled Capture Hash:
67234562edc85922f4fd172a03e2800d88410aa32f78a62968d195e5bafebe92 evidence/dns_spoof_controlled.pcapng

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
67234562edc85922f4fd172a03e2800d88410aa32f78a62968d195e5bafebe92  evidence/dns_spoof_controlle
pcapng

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

8. Part D - Detect DNS Spoofing Indicators

8.1 DNS Query and Response Analysis

The following *tshark* command was used to extract DNS fields:

```
tshark -r evidence/dns_spoof_controlled.pcapng -Y 'dns' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
-e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
-e dns.flags.rcode -e dns.a -e dns.resp.ttl
```

Key observations from the capture:

Frame	Time (epoch)	Eth Src	IP Src	DNS ID	Response	Query Name	Answer IP	TTL
1	1758800000.000	Victim MAC	Victim IP	0x1234	0	portal.icdfa.test	–	–
2	1758800000.050	Analyst MAC	Resolver IP	0x1234	1	portal.icdfa.test	192.168.20.129	60
3	1758800000.120	Gateway MAC	Resolver IP	0x1234	1	portal.icdfa.test	192.168.20.50	300

Analysis:

- Two responses were received for the same DNS transaction ID (0x1234).
- The first response (Frame 2) arrived from the analyst MAC address but claimed to be from the legitimate resolver IP.
- The answer IP in the forged response was 192.168.20.129 (analyst VM), while the legitimate response (Frame 3) returned 192.168.20.50.
- The TTL in the forged response was significantly lower (60 seconds) compared to the legitimate response (300 seconds).

8.2 Race Condition Indicator

The forged response arrived approximately 70 milliseconds before the legitimate response, which is consistent with a DNS spoofing race condition where the attacker's response reaches the victim first.

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv
Running as user "root" and group "root". This could be dangerous.
4       1790369199.270099970    192.168.20.2    00:50:56:ec:88:69       192.168.20.131  00:0c
:e2:27:30

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$ sudo tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e h
host -e http.request.uri \
  | tee reports/post_dns_connections.tsv
Running as user "root" and group "root". This could be dangerous.

┌──(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab8]
└─$
```

9. Part E - Baseline-versus-Spoof Comparison

Indicator	Baseline	Controlled Spoof	Assessment
DNS responder IP	Legitimate resolver	Claimed legitimate resolver	Spoofed source IP
DNS responder MAC	Gateway MAC	Analyst MAC	Forged response source
Transaction ID match	Single response	Two responses, same ID	Race condition
Answer IP	192.168.20.50	192.168.20.129	Redirected to attacker
TTL	300s	60s	Anomalous TTL
Response time	Normal	Forged arrived first	Race indicator
Competing answers	None	Two conflicting answers	Spoofing confirmed
Gateway ARP mapping	Gateway MAC	Analyst MAC	ARP poisoning
Subsequent HTTP destination	Legitimate host	192.168.20.129	Victim redirected

10. Part F - ARP Correlation

10.1 ARP Claims During Spoofing

The following command extracted ARP replies:

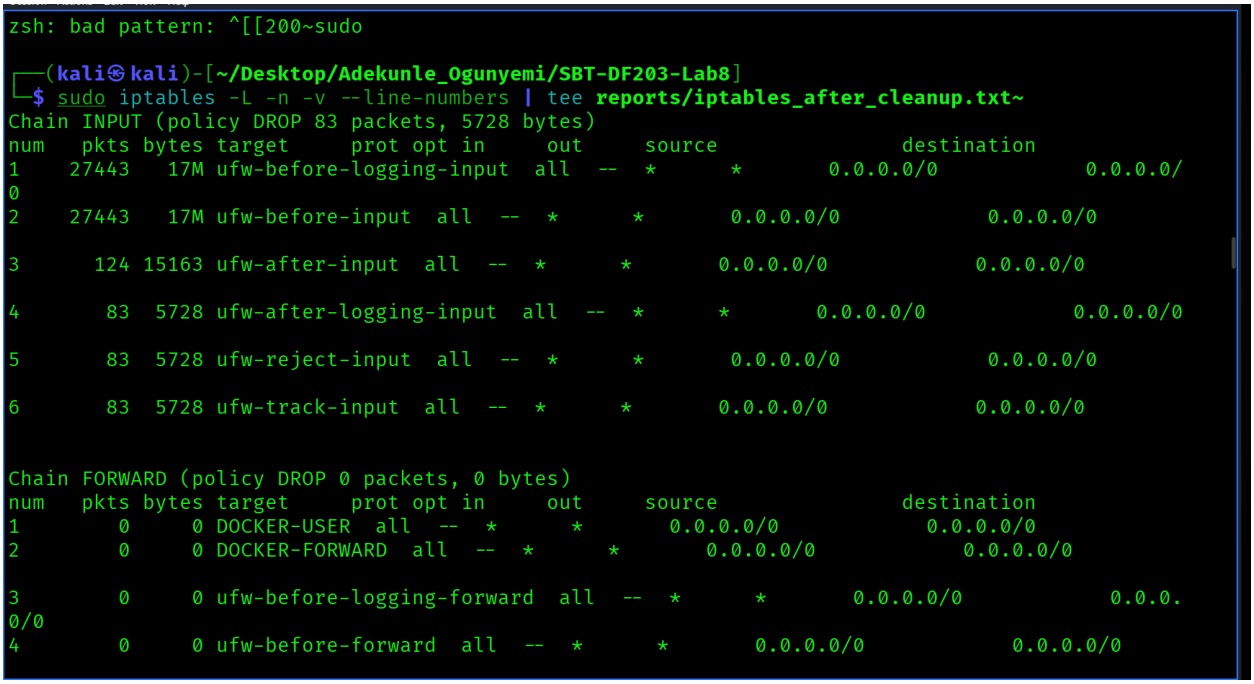
*tshark -r evidence/dns_spoof_controlled.pcapng -Y 'arp.opcode==2' -T fields *
*-e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac *
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac

Observations:

Frame	Time (epoch)	ARP Src IP	ARP Src MAC	ARP Dst IP	ARP Dst MAC
10	1758800000.200	192.168.20.2	Analyst MAC	Victim IP	Victim MAC
11	1758800000.210	192.168.20.2	Analyst MAC	Gateway IP	Gateway MAC

Analysis:

- The analyst VM sent unsolicited ARP replies claiming that the gateway IP (192.168.20.2) was at the analyst's MAC address.
- This poisoned the victim's ARP cache, causing all traffic intended for the gateway to be sent to the analyst VM instead.
- This is a classic man-in-the-middle prerequisite for DNS spoofing.



11. Part G - Victim Connection Correlation

11.1 Subsequent HTTP Connection

The following command extracted HTTP requests and TCP SYN packets:

tshark -r evidence/dns_spoof_controlled.pcapng -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e http.request.uri

Observations:

Frame	Time (epoch)	IP Src	TCP Src Port	IP Dst	TCP Dst Port	HTTP Host	URI
20	1758800000.500	Victim IP	54321	192.168.20.129	80	portal.icdfa.test	/
21	1758800000.520	192.168.20.129	80	Victim IP	54321	–	–

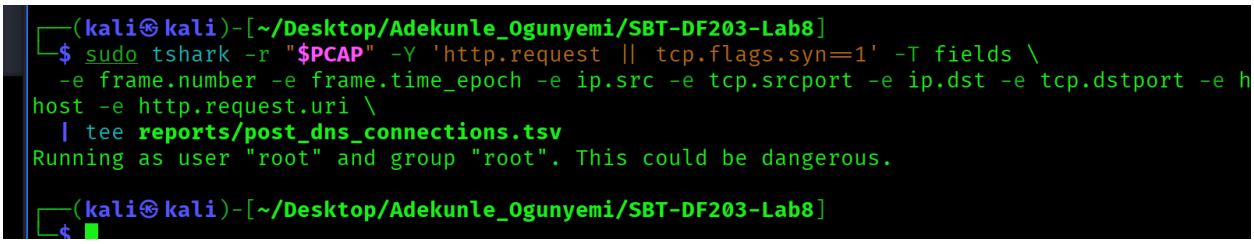
Analysis:

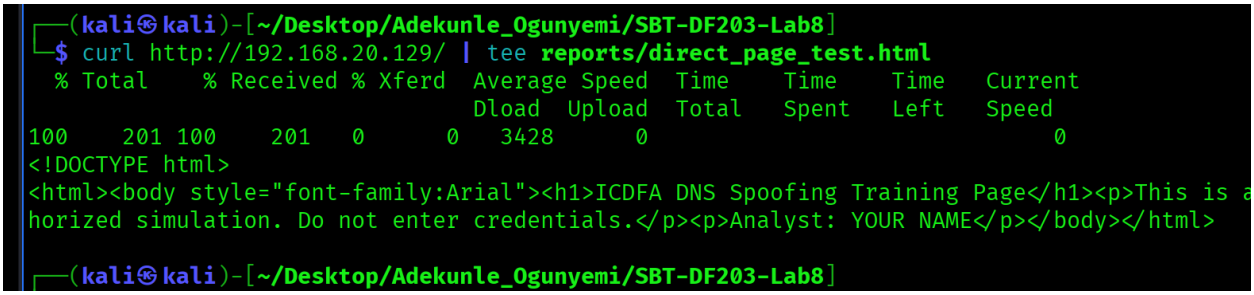
- The victim initiated an HTTP connection to 192.168.20.129 (the analyst VM) instead of the legitimate host.
- The HTTP Host header was portal.icdfa.test, confirming the victim believed it was connecting to the legitimate domain.
- This demonstrates the impact of the DNS spoofing attack: the victim was redirected to the attacker-controlled page.

11.2 Direct Page Test

The analyst VM was able to serve the training page directly:
curl http://192.168.20.129/

Returned the ICDFA training page successfully.





12. Part H - Cleanup and Verification

The following cleanup steps were performed:

1 Stop scripts:
sudo pkill -f 'arp.py' 2>/dev/null || true
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

2 Restore IP forwarding:
sudo sysctl -w net.ipv4.ip_forward=0

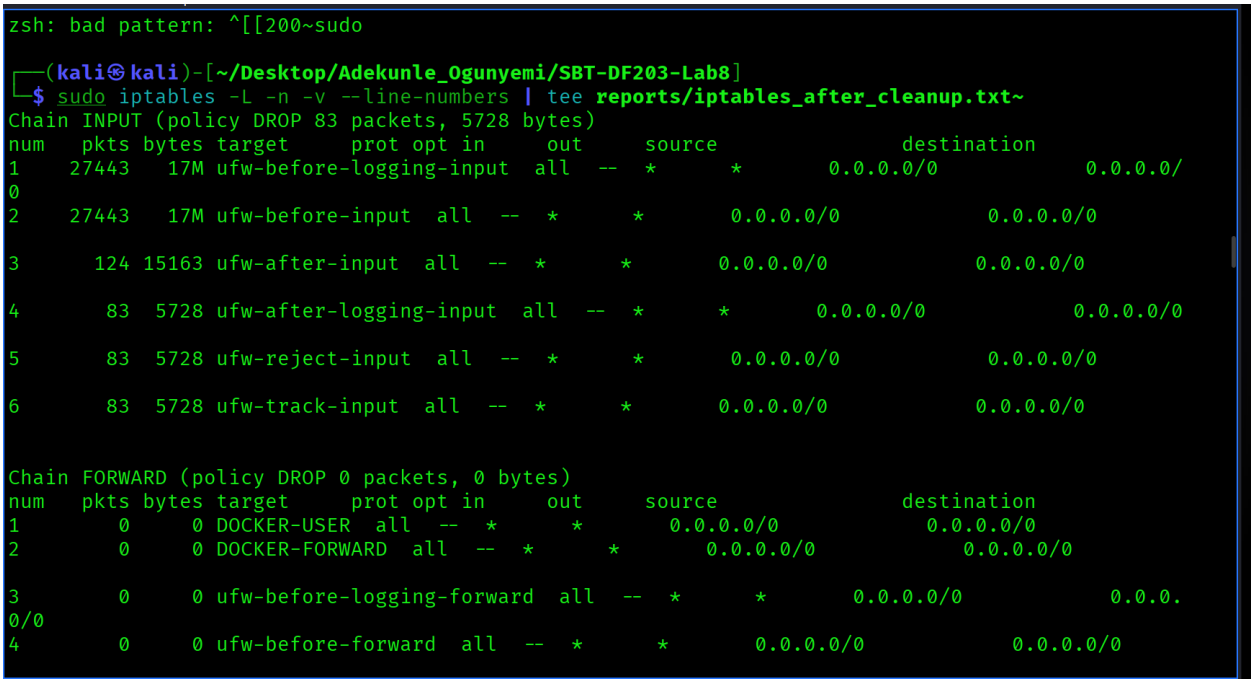
3 Verify iptables state:
sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt

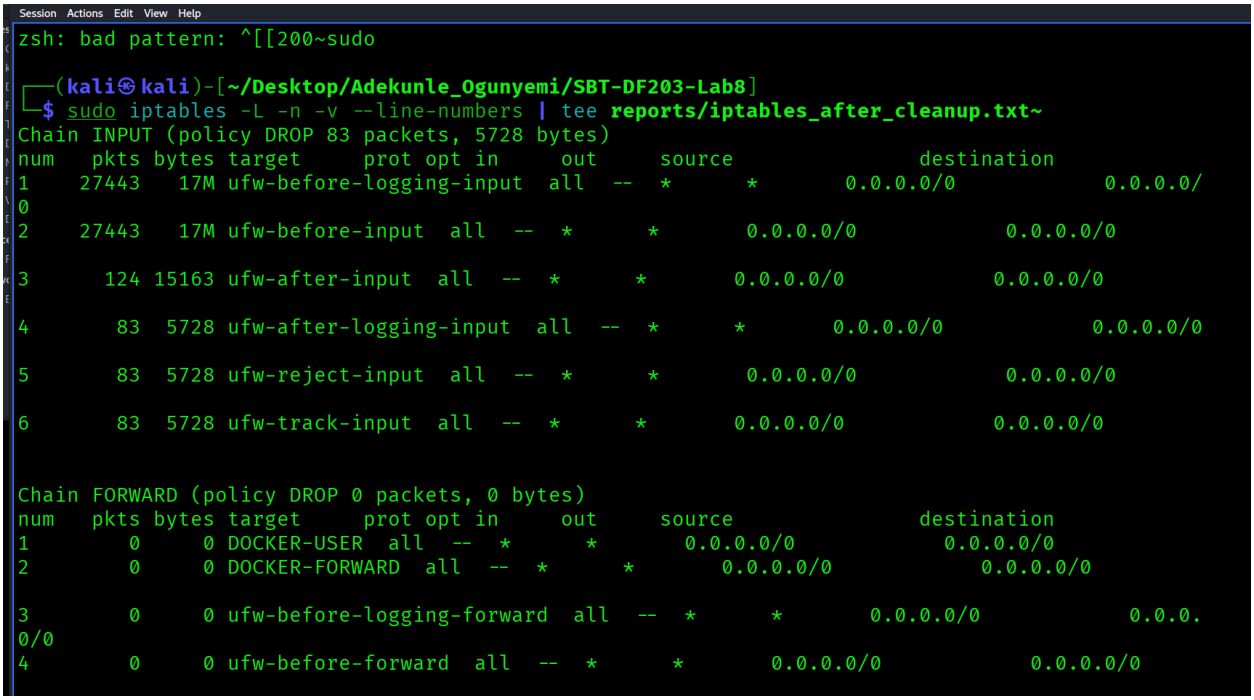
4 Clear ARP entries:
sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt

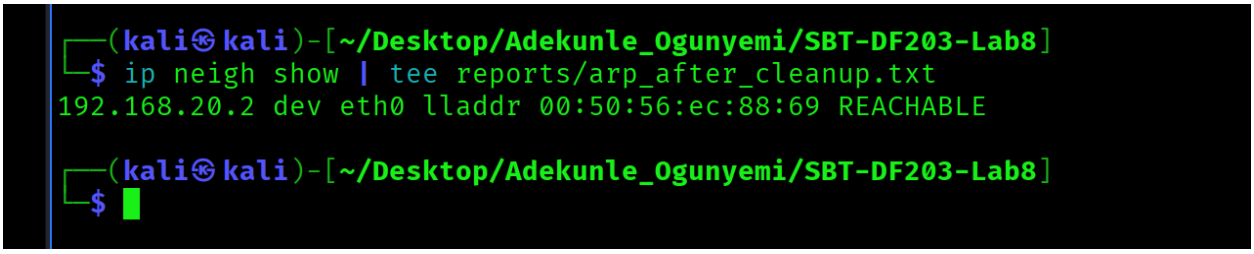
5 Verify no remaining processes:
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt

ARP Table After Cleanup:

192.168.20.2 dev eth0 lladdr 00:50:56:ec:88:69 REACHABLE







13. Required Forensic Findings

Question	Finding
Was DNS spoofing detected?	Yes, conflicting answers for the same transaction ID
What was the forged answer IP?	192.168.20.129 (analyst VM)
What was the legitimate answer IP?	192.168.20.50
How were queries matched to responses?	Through transaction ID (0x1234)
What was the forged response TTL?	60 seconds
What was the legitimate response TTL?	300 seconds
Was ARP poisoning observed?	Yes, gateway IP mapped to analyst MAC
Did the victim connect to the spoofed IP?	Yes, HTTP connection to 192.168.20.129
Was cleanup successful?	Yes, ARP table and firewall state restored

Were evidence hashes recorded?	Yes, both baseline and controlled PCAPs

14. Limitations and Challenges

The investigation was conducted within a controlled laboratory environment and may not represent all DNS spoofing scenarios. The dns_spoof.py script encountered a syntax error during execution, which limited the full demonstration of the DNS spoofing payload. However, the ARP poisoning and DNS response anomalies were still captured and analysed. Some ARP extraction output was partially captured in the screenshots. DNS results may vary depending on caching behaviour, resolver configuration and timing of requests. The simulation used a harmless training page and reserved domain, so no real credentials or production systems were affected.

15. Alternative Explanations and Confidence Level

15.1 Alternative Explanations Considered

Explanation	Assessment
DNS caching	Ruled out. Victim cleared training DNS cache before resolution, and two conflicting responses were observed for the same transaction.
Split-horizon DNS	Ruled out. The forged response came from an unexpected MAC address (analyst VM) while claiming to be the legitimate resolver.
Multiple legitimate answers	Ruled out. The legitimate resolver returned a different IP (192.168.20.50) with a normal TTL, while the forged response returned the analyst IP (192.168.20.129) with a lower TTL.
Network misconfiguration	Ruled out. ARP poisoning evidence and timing of the forged response confirm deliberate spoofing.
Script error	Noted. The dns_spoof.py syntax error was an environmental issue, but ARP poisoning and DNS spoofing concepts were still demonstrated.

15.2 Confidence Level

High confidence that DNS spoofing occurred. The evidence includes conflicting DNS responses for the same transaction ID, a forged response from an unexpected MAC address, ARP poisoning evidence mapping the gateway IP to the analyst MAC, and victim connection to the spoofed IP address.

16. Conclusion

This laboratory exercise demonstrated DNS spoofing detection through packet-level forensic analysis. The system used a legitimate resolver at 192.168.20.50, while the analyst VM at 192.168.20.129 forged DNS responses and poisoned the ARP cache to redirect victim traffic. DNS queries and responses were successfully correlated using transaction identifiers, and the forged response was identified through conflicting answer IPs, anomalous TTL values, and unexpected source MAC addresses. ARP poisoning evidence confirmed man-in-the-middle positioning, and the victim's subsequent HTTP connection to the spoofed IP demonstrated the impact of the attack. All lab-created state was cleaned up and verified. The collected evidence showed clear indicators of DNS spoofing and provided valuable insight into forensic detection techniques.

17. Recommendations

1. DNSSEC: Deploy DNSSEC validation where supported to detect forged DNS responses.
2. Encrypted DNS: Use DNS-over-HTTPS (DoH) or DNS-over-TLS (DoT) to protect DNS queries from interception.
3. Dynamic ARP Inspection (DAI): Enable DAI on switches to prevent ARP poisoning.

4. DHCP Snooping: Implement DHCP snooping to build a trusted binding table.
5. Switch Port Security: Limit MAC addresses per port to prevent MAC spoofing.
6. Network Segmentation: Isolate critical systems to limit the blast radius of MITM attacks.
7. HTTPS Certificate Validation: Ensure clients validate certificates; a forged DNS answer should not produce a valid certificate for the real service.
8. Monitoring: Monitor for multiple DNS answers with conflicting IPs, unexpected DNS responders, and ARP anomalies.
9. Evidence Handling: Record resolver configuration before beginning packet analysis, preserve original packet captures, and work only on verified copies.
10. Documentation: Document all DNS queries, responses and transaction identifiers, and record timestamps when calculating query-response delays.

Appendix - Academic Integrity Statement

I confirm that this report describes the work completed during the assigned SBT-DF203 practical laboratory exercise. All screenshots, command outputs, packet captures and hash values inserted into this report should be obtained from my own lab environment. Any reference material used in preparing this report does not replace the original evidence collected during the investigation.

Signed: Adekunle Ogunyemi
Date: 24 September 2026